

TP-docker-K8S CARLOT Steven DE1

Build de l'image

```
> docker build -t mon-service:1.0 .
[+] Building 22.8s (10/10) FINISHED
docker:default
=> [internal] load build definition from Dockerfile
0.0s
=> => transferring dockerfile: 182B
0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim
0.5s
=> [internal] load .dockerignore
0.0s
=> => transferring context: 2B
0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
15.5s
=> => resolve docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
0.1s
=> =>
sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca
249B / 249B
0.2s
=> =>
sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf0
12.12MB / 12.12MB
7.5s
=> =>
sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70
1.29MB / 1.29MB
4.3s
=> =>
sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8
29.83MB / 29.83MB
13.8s
=> => extracting
sha256:6b37362b3da78869050b894b799ad4df04f1f3b52774087db0d81151570244c8
0.9s
=> => extracting
sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70
0.1s
=> => extracting
sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf0
0.4s
=> => extracting
sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca
```

```

0.0s
=> [internal] load build context
0.1s
=> => transferring context: 982B
0.0s
=> [2/5] WORKDIR /app
0.1s
=> [3/5] COPY requirements.txt .
0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
5.4s
=> [5/5] COPY . .
0.1s
=> exporting to image
0.8s
=> => exporting layers
0.5s
=> => exporting manifest
sha256:f970c4b5597cdf816873e2cae826c9aa6cca96691084d1461b85d52e6095d3b2
0.0s
=> => exporting config
sha256:187b43991d3f4eb1e714257539881f142a3a3e71ea7f3b8df514e719d2722683
0.0s
=> => exporting attestation manifest
sha256:89dacf4d0d9614896f07997746fd68d99a9bd474b969aedbd3de05c68a0b5322
0.0s
=> => exporting manifest list
sha256:d8cebe1d197094530e2615e303a4f248833120de53d73ab92cdb272a3159f324
0.0s
=> => naming to docker.io/library/mon-service:1.0
0.0s
=> => unpacking to docker.io/library/mon-service:1.0
0.2s

```

Démarrez l'ensemble :

```

✓ Image postgres:16                Pulled
58.6s
✓ Image tp-docker-k8s-api           Built
1.5s
✓ Volume tp-docker-k8s_donnees_db  Created
0.0s
✓ Network tp-docker-k8s_default    Created
0.1s
✓ Container tp-docker-k8s-db-1     Started
0.5s
✓ Container tp-docker-k8s-api-1    Started

```

Verification

```
curl http://localhost:8000/health  
{ "status": "ok" }
```

```
curl http://localhost:8000/ventes  
[[1, "Abonnement A", "120.00"], [2, "Abonnement B", "89.90"]]
```

```
• > curl http://localhost:8000/ventes  
[[1, "Abonnement A", "120.00"], [2, "Abonnement B", "89.90"]]
```

4 Vérifier la persistance des données

Étape 4

```
docker compose down  
docker compose up -d  
curl http://localhost:8000/ventes
```

```
• > curl http://localhost:8000/ventes  
[[1, "Abonnement A", "120.00"], [2, "Abonnement B", "89.90"]]
```

5 Installer et démarrer un cluster Kubernetes local

Étape 5

```
sudo pacman -S minikube kubectl
```

5.2 — Vérifier les installations

```
• > minikube version  
minikube version: v1.38.1  
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0-dirty  
  
◉ > kubectl version  
Client Version: v1.37.0  
Kustomize Version: v5.8.1
```

5.3 — Démarrer le cluster

```

• > minikube start --driver=docker
🌟 minikube v1.38.1 sur Cachyos
🚀 Utilisation du pilote docker basé sur la configuration de l'utilisateur
! Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔧 Utilisation du pilote Docker avec le privilège root
👍 Démarrage du nœud "minikube" primary control-plane dans le cluster "minikube"
📦 Extraction de l'image de base v0.0.50...
📦 Téléchargement du préchargement de Kubernetes v1.35.1...
> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 9.48 Mi
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 2.58 Mi
🔥 Création de docker container (CPU=2, Memory=5900Mo) ...
🐳 Préparation de Kubernetes v1.35.1 sur Docker 29.2.1...
🔗 Configuration de bridge CNI (Container Networking Interface)...
🔍 Vérification des composants Kubernetes...
  • Utilisation de l'image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Modules activés: storage-provisioner, default-storageclass

! /usr/bin/kubectl est la version 1.37.0, qui peut comporter des incompatibilités avec Kubernetes 1.35.1.
  • Vous voulez kubectl v1.35.1 ? Essayez 'minikube kubectl -- get pods -A'
🎉 Terminé ! kubectl est maintenant configuré pour utiliser "minikube" cluster et espace de noms "default" par défaut.

```

```

• > kubectl get nodes

```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	4m31s	v1.35.1

Étape 6

6.4 — Appliquer les manifests

```

• > kubectl apply -f deployment.yaml
deployment.apps/api-deployment created

```

```

• > kubectl apply -f service.yaml
service/api-service created

```

7 Observer la résilience et la montée en charge

7.1 — Auto-réparation

Commande:

```

kubectl get pods
kubectl delete pod api-deployment-7689b7bfb-nqqh5
kubectl get pods

```

```

• > kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
api-deployment-7689b7bfb-nqqh5     1/1     Running   0           4m51s
api-deployment-7689b7bfb-nwq4g     1/1     Running   0           4m51s
api-deployment-7689b7bfb-sv4r6     1/1     Running   0           4m51s
• > kubectl delete pod api-deployment-7689b7bfb-nqqh5
pod "api-deployment-7689b7bfb-nqqh5" deleted from default namespace
• > kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
api-deployment-7689b7bfb-79l6b     1/1     Running   0           2m38s
api-deployment-7689b7bfb-nwq4g     1/1     Running   0           8m37s
api-deployment-7689b7bfb-sv4r6     1/1     Running   0           8m37s

```

7.2 — Montée en charge

Commande:

```

kubectl scale deployment api-deployment --replicas=5
kubectl get pods

```

```

• > kubectl scale deployment api-deployment --replicas=5
kubectl get pods
deployment.apps/api-deployment scaled
NAME                                READY   STATUS              RESTARTS   AGE
api-deployment-7689b7bfb-562rx     0/1     ContainerCreating   0           1s
api-deployment-7689b7bfb-79l6b     1/1     Running             0           5m
api-deployment-7689b7bfb-nwq4g     1/1     Running             0           10m
api-deployment-7689b7bfb-r76q7     0/1     ContainerCreating   0           1s
api-deployment-7689b7bfb-sv4r6     1/1     Running             0           10m

```

8 Versionner le projet

```

> git add api/Dockerfile docker-compose.yml deployment.yaml service.yaml
api/
> git commit -m "TP Jour 2 : containerisation et deploiement Kubernetes"
[main 87256a5] TP Jour 2 : containerisation et deploiement Kubernetes
7 files changed, 100 insertions(+)
create mode 100644 api/Dockerfile
create mode 100644 api/app.py
create mode 100644 api/init.sql
create mode 100644 api/requirements.txt
create mode 100644 deployment.yaml
create mode 100644 docker-compose.yml
create mode 100644 service.yaml
> git push
Énumération des objets: 11, fait.
Décompte des objets: 100% (11/11), fait.

```

```
Compression par delta en utilisant jusqu'à 8 fils d'exécution
Compression des objets: 100% (9/9), fait.
Écriture des objets: 100% (10/10), 1.77 KiO | 603.00 KiO/s, fait.
Total 10 (delta 0), réutilisés 0 (delta 0), réutilisés du paquet 0 (depuis 0)
To https://github.com/S13v3n-2/tp-docker-k8s.git
ccfe286..87256a5  main -> main
```

9 Questions de compréhension

- 1. **Quelle est la différence entre une image Docker et un conteneur Docker ?** Une image Docker est un modèle (un plan figé contenant le code, les bibliothèques et la configuration), tandis qu'un conteneur Docker est une instance vivante et isolée en cours d'exécution de cette image .
- 2. **Pourquoi les données de la base sont-elles perdues si l'on supprime le volume, mais pas si l'on supprime seulement le conteneur ?** Parce que le cycle de vie d'un volume Docker est totalement indépendant de celui du conteneur. Les données y sont stockées directement sur le système de fichiers de la machine hôte, en dehors de la couche éphémère du conteneur.
- 3. **Un Deployment Kubernetes déclaré avec replicas: 3 perd un pod. Que fait Kubernetes, et pourquoi parle-t-on d'approche déclarative ?** Le contrôleur de réplication détecte que le nombre de pods actifs est tombé à deux, compare cet état à la consigne demandée (replicas: 3), et planifie immédiatement la création d'un nouveau pod pour rétablir le compte demandé. On ne donne pas d'ordres séquentiels au système, mais on déclare l'état cible souhaité. Kubernetes applique une boucle de reconstruction continue pour aligner en permanence l'état réel sur cet état désiré déclaré dans la configuration.
- 4. **Quel est le rôle du Service par rapport aux pods, sachant que ces derniers sont recréés régulièrement avec une nouvelle adresse IP ?** Le Service fournit une adresse IP virtuelle fixe, cela est indépendants du cycle de vie des pods. Via les sélecteurs d'étiquettes (labels/selectors), il met à jour automatiquement la liste des pods sains cibles (Endpoints) et répartit la charge entre eux, masquant ainsi totalement les changements d'adresses IP des pods.
- 5. **Citez deux limites de Docker seul (sans Kubernetes) que vous avez observées ou apprises ce matin.** Si un conteneur plante ou qu'un nœud tombe en panne, Docker ne recrée pas automatiquement de nouvelle instance sur une autre machine pour maintenir un nombre cible de copies en direct. Docker standard opère sur un seul serveur hôte. Il n'intègre pas nativement la répartition de conteneurs sur un cluster de machines, ni la gestion dynamique du trafic réseau distribué entre plusieurs serveurs physiques.